

Lab 1 ve Lab 2 Teknik Raporu

Başlık: Linux Sunucu Güvenlik Değerlendirmesi ve Sıkılaştırma

Alan: Güvenlik Mühendisliği, Linux Hardening, Altyapı Savunması

Ortam: AWS EC2 - Ubuntu 24.04.4 LTS

Kapsam: Mega-Lab 1 (Güvenlik Değerlendirmesi ve Tehdit Keşfi) ve Mega-Lab 2 (Firewalling, SSH Security ve Linux Hardening)

Amaç: Bu doküman, Lab 1 ve Lab 2 boyunca yapılan tüm teknik çalışmayı tek bir kapsamlı raporda toplar. İçeriğe değerlendirme mantığı, komutların amacı, sahadan elde edilen kanıtlar, özel baseline scripti, bulgular, tehdit modelleme, hardening adımları ve doğrulama sonuçları dahildir. Hedef, bu dokümanın ek ekran görüntülerine ihtiyaç duymadan tek başına portfolyo düzeyinde bir teknik kanıt olarak kullanılabilmesidir.

MODULE 03 Security Engineering & Incident Management
Security Foundations & Risk Management - The CIA triad, authentication, authorization, accountability, assets, threats, vulnerabilities, risk, attack surface, and defense in depth.
Threat Modeling & Attack Lifecycle - Trust boundaries, data-flow diagrams, STRIDE, attack trees, the Cyber Kill Chain, MITRE ATT&CK, and the attack lifecycle.
Network and Availability Threats - MITM, spoofing, session hijacking, DNS and ARP attacks, DoS/DDoS, SYN floods, and service-disruption scenarios.
Malware & Endpoint Defense - Worms, Trojans, ransomware, rootkits, botnets, fileless malware, sandboxing, hashes, EDR, and endpoint hardening.
Firewalling & Network Defense - Stateful and stateless filtering, Netfilter, iptables, nftables, connection tracking, NAT, rate limiting, and segmentation.
Cryptography, PKI & SSH Security - Symmetric and asymmetric cryptography, hashing, digital signatures, certificates, SSH key pairs, host keys, and sshd_config hardening.
Linux Hardening & Vulnerability Management - Attack-surface reduction, CIS Benchmarks, CVE/CVSS, patching, service inventory, permissions, PAM, SELinux, and AppArmor fundamentals.
System Auditing, Logging & SIEM - /var/log, the systemd journal, auditd, log integrity, centralized logging, SIEM, detection rules, and security-event analysis.
Log-Based Abuse Mitigation - Fail2Ban behavior, authentication-log analysis, temporary blocking, rate limiting, and the operational limitations of log-based controls.
Incident Response Fundamentals - Preparation, detection, triage, containment, eradication, recovery, evidence preservation, communication, and post-incident review.

Şekil 1. Projenin kavramsal çerçevesi olarak kullanılan Module 03 konu haritası.

1. Yönetici Özeti

Bu proje, AWS EC2 üzerinde çalışan bir Ubuntu sunucusunun uçtan uca güvenlik değerlendirme ve sıkılaştırılmasını dokümante eder. Çalışma; işletim sistemi, kernel, yerel kullanıcılar, sudo yetkileri, SSH yapılandırması, dinleyen portlar, çalışan servisler, firewall durumu, paket güncelleme durumu ve başarısız kimlik doğrulama sinyallerinin keşfiyle

başladı. Elde edilen baseline, daha sonra kontrollü hardening adımlarına temel oluşturdu ve gereksiz maruziyet azaltılırken erişilebilirlik ile izlenebilirlik korunmuş oldu.

Çalışma şu mühendislik kalıbını izledi:

Threat -> Control -> Validation

Yani sistem üzerinde rastgele değişiklik yapmak yerine, her sıkılaştırma adımı somut bir bulguya dayandırıldı ve daha sonra teknik kanıtla doğrulandı.

2. Projenin Hedefleri

- Bir Linux sunucusu için tekrar üretilebilir bir baseline assessment süreci oluşturmak.
- Host seviyesinden tespit edilebilen güvenlik zafiyetlerini belirlemek.
- Kanıt toplama sürecini otomatikleştiren tekrar kullanılabilir bir Bash scripti geliştirmek.
- Expose edilen servisleri, kimlik risklerini ve SSH güvenlik duruşunu değerlendirmek.
- Saldırı yüzeyini küçültmek için hedefli hardening kontrolleri uygulamak.
- Tüm değişiklikleri before/after kontrolleriyle doğrulamak.
- Profesyonel GitHub portfolyosuna uygun teknik bir döküman üretmek.

3. Ortam ve Operasyonel Bağlam

Bileşen	Değer
Bulut Sağlayıcı	AWS
Hesaplama Servisi	EC2
İşletim Sistemi	Ubuntu 24.04.4 LTS
Kernel	Linux 7.0.0-1009-aws
Donanım Modeli	t3.micro
Mimari	x86_64
Uzaktan Yönetim	SSH
Host Firewall	UFW
SSH Koruması	Fail2Ban
Tespit Edilen Ek Public Servis	80 portunda Nginx

4. Modül Kapsamı

Bu rapordaki pratik çalışma Module 03 içindeki şu başlıklarla eşleşmektedir:

- Security Foundations & Risk Management
- Threat Modeling & Attack Lifecycle

- Network and Availability Threats
- Firewalling & Network Defense
- Cryptography, PKI & SSH Security
- Linux Hardening & Vulnerability Management
- System Auditing, Logging & SIEM hazırlığı
- Log-Based Abuse Mitigation hazırlığı

Bu doküman özellikle şu iki pratik fazı kapsar:

Lab	Odak	Durum
Lab 1	Güvenlik Değerlendirmesi, Asset Discovery, Threat Discovery	Tamamlandı
Lab 2	Linux Hardening, SSH Hardening, Service Reduction, Firewall Cleanup	Tamamlandı

5. Metodoloji

5.1 Mühendislik Mantığı

Hardening fazındaki her kontrol, önceki bir gözleme bağlandı. Metodoloji özellikle düzensiz deneme-yanılma yaklaşımından kaçınacak şekilde tasarlandı. İzlenen akış şuydu:

```
Sistem Kimliği
-> Kullanıcı ve Yetki Analizi
-> SSH Yapılandırma İncelemesi
-> Ağ Maruziyeti İncelemesi
-> Servis Envanteri
-> Firewall İncelemesi
-> Kimlik Doğrulama Sinyallerinin İncelenmesi
-> Baseline Otomasyonu
-> Tehdit Modelleme
-> Kontrollü Hardening
-> Doğrulama
```

5.2 Threat - Control - Validation Modeli

Threat	Control	Validation
Gereksiz şekilde ağa açık servis	Servisi durdurmak ve firewall kuralını kaldırmak	Servisin inactive olduğunu ve portun artık expose olmadığını doğrulamak
Aktif parolaya sahip kullanılmayan yerel hesap	Hesabı kilitlemek	P durumunun L durumuna geçtiğini kontrol etmek
Gereksiz SSH özelliğinin açık olması	X11 forwarding'i kapatmak	sshd -T ile efektif SSH yapılandırmasını doğrulamak
Zayıf dosya sahipliği veya izin modeli	Kısıtlayıcı izinler vermek ve doğru sahipliği atamak	Hedef dizini yeniden listeleyip mode ve owner bilgisini doğrulamak

6. Lab 1 - Baseline Güvenlik Değerlendirmesi

6.1 Amaç

Lab 1'in amacı, savunma değişiklikleri yapmadan önce sunucunun gerçek durumunu anlamaktır. Bu faz şu sorulara cevap verir: sistemde neler var, neler expose, kimler privileged ve hangi alanlar risklidir.

6.2 Sistem Kimliğinin Toplanması

İlk kullanılan komutlar şunlardı:

```
uname -a
hostnamectl
```

Bu komutlar; işletim sistemi, kernel satırı, mimari, sanallaştırma bağlamı ve EC2 donanım profilini anlamak için kullanıldı.

Gözlemlenen kanıt:

```
Operating System: Ubuntu 24.04.4 LTS
Kernel: Linux 7.0.0-1009-aws
Architecture: x86_64
Virtualization: amazon
Hardware Vendor: Amazon EC2
Hardware Model: t3.micro
```

Bu çıktı, hedefin yerel bir sanal makine ya da container değil, AWS üzerinde çalışan bir Ubuntu EC2 sunucusu olduğunu doğruladı.

6.3 Mevcut Kullanıcı ve Yetki İncelemesi

Yönetimsel yetkiyi anlamak için şu komutlar kullanıldı:

```
whoami
id
sudo -l
```

Gözlemlenen kanıt:

```
ubuntu
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),4(adm),24(cdrom),27(sudo),
30(dip),105(lxd)

User ubuntu may run the following commands on ip-172-31-42-76:
(ALL : ALL) ALL
(ALL) NOPASSWD: ALL
```

Bu, `ubuntu` hesabının parola sormayan tam sudo yetkisine sahip olduğunu gösterdi. Bulut imajlarında bu durum yaygın olsa da güvenlik açısından kritik bir kimliktir; hesap ya da SSH anahtarı ele geçirilirse saldırgan doğrudan root seviyesinde kontrol elde edebilir.

6.4 Yerel Hesap Envanteri

Yerel hesaplar şu komutla listelendi:

```
awk -F: '$3 >= 1000 {print $1, $3, $6, $7}' /etc/passwd
```

Gözlemlenen kanıt:

```
nobody 65534 /nonexistent /usr/sbin/nologin
ubuntu 1000 /home/ubuntu /bin/bash
dev-user 1001 /home/dev-user /bin/bash
```

Bu çıktı, ek bir yerel kullanıcı olan `dev-user` hesabını ortaya çıkardı ve bu hesabın ayrıca incelenmesi gerektiğini gösterdi.

Takip komutları:

```
sudo groups dev-user
sudo passwd -S dev-user
sudo -l -U dev-user
```

Gözlemlenen kanıt:

```
dev-user : dev-user users
dev-user P 2026-07-30 0 99999 7 -1

User dev-user may run the following commands on ip-172-31-42-76:
(ALL) NOPASSWD: /usr/bin/systemctl status *
```

Bu hesap tam yetkili yönetici değildi; ancak aktif parola durumu ve sınırlı da olsa sudo yetkisi nedeniyle güvenlik açısından anlamlı bir kimlik riski oluşturuyordu.

6.5 SSH Güvenlik İncelemesi

Efektif SSH yapılandırması şu komutla kontrol edildi:

```
sudo sshd -T | grep -E '^(permitrootlogin|passwordauthentication|
pubkeyauthentication|kbdinteractiveauthentication|maxauthtries|allowusers|
peremptypasswords|x11forwarding|port)'
```

Gözlemlenen kanıt:

```
port 22
maxauthtries 3
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
x11forwarding yes
peremptypasswords no
```

Bazı güvenli varsayılanlar zaten mevcuttu: doğrudan root SSH girişi kapalıydı, parola tabanlı giriş kapalıydı, keyboard-interactive kimlik doğrulama kapalıydı ve public-key authentication açıktı. Buna karşın `X11Forwarding` headless bir EC2 sunucusunda gereksizdi ve operasyonel bir fayda sağlamadan SSH özellik kümesini genişletiyordu.

6.6 Servis ve Ağ Maruziyeti İncelemesi

Dinleyen servisler şu komutla incelendi:

```
sudo ss -tlnp
```

Öne çıkan maruziyetler şunlardı:

```
0.0.0.0:80      nginx
0.0.0.0:22      sshd
[::]:80        nginx
[::]:22        sshd
```

Bu, hem SSH hem de Nginx servislerinin tüm ağ arayüzlerinde erişilebilir olduğunu gösterdi. Özellikle 80 portunda çalışan Nginx önemlidir; çünkü bu lab aşamasında kasıtlı olarak yayınlanan bir web uygulaması tanımlı değildi.

Yerel web sunucu davranışını doğrulamak için şu komut kullanıldı:

```
curl -I localhost
```

Gözlemlenen kanıt:

```
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Wed, 05 Aug 2026 10:54:03 GMT
Content-Type: text/html
Content-Length: 615
```

Bu da Nginx'in sadece kurulu olmadığını, gerçekten içerik servis ettiğini kanıtladı.

6.7 Firewall İncelemesi

Host firewall kuralları şu komutla incelendi:

```
sudo ufw status numbered
```

Temizlik öncesi gözlemlenen kanıt:

```
Status: active

[ 1] 22/tcp          ALLOW IN    Anywhere
[ 2] 80/tcp           ALLOW IN    Anywhere
[ 3] 22/tcp (v6)      ALLOW IN    Anywhere (v6)
[ 4] 80/tcp (v6)      ALLOW IN    Anywhere (v6)
```

Inbound default-deny durumu faydalıydı; fakat buna rağmen 80 portu için açık allow kuralı Nginx servisini internete expose etmeye devam ediyordu.

6.8 Kimlik Doğrulama Sinyallerinin İncelenmesi

Kimlik doğrulama ile ilgili kanıtlar, internete açık SSH servisinin hâlihazırda tarandığını gösterdi. İlgili örnekler auth log içinde invalid-user ya da malformed connection benzeri olaylardı. Buna ek olarak, daha sonra bakılan Fail2Ban çıktısı da SSH jail mantığının daha önce bazı aktiviteleri gördüğünü ortaya koydu.

6.9 Neden Baseline Scripti Oluşturuldu

Bu noktada her değerlendirme komutunu elle tekrar etmek hem zaman kaybettirici hem de hataya açıktı. Bu nedenle en kritik güvenlik sinyallerini tek çalıştırmada toplayan ve sonucu zaman damgalı rapor dosyasına yazan read-only bir otomasyon scripti geliştirildi.

7. Özel Script - `ec2_security_baseline_check.sh`

7.1 Scriptin Amacı

Bu script, sistem durumunu değiştirmeden host seviyesinde kanıt toplamayı otomatikleştirmek için tasarlandı. Linux güvenlik incelemelerinde tekrar kullanılabilir bir baseline assessment aracı olarak düşünüldü.

- Kimlik ve yetki bilgilerini envanterler.
- Efektif SSH durumunu kontrol eder.
- Dinleyen servisleri ve çalışan unit'leri listeler.
- Firewall durumunu kontrol eder.
- Bekleyen güvenlik güncellemelerini arar.
- Son başarısız SSH ile ilişkili log aktivitelerini inceler.
- Tüm bilgiyi zaman damgalı bir rapor dosyasına kaydeder.

7.2 Scriptin Tam Hali

```
#!/bin/bash

# =====
# EC2 Security Baseline Assessment Script
# Version : 1.0
# Purpose : Read-only security inventory - no changes made
# =====

REPORT="$HOME/baseline_report_$(date +%Y%m%d_%H%M%S).txt"

log() {
    echo -e "$1" | tee -a "$REPORT"
}

log "=====
log " EC2 SECURITY BASELINE REPORT"
```

```

log " Generated : $(date)"
log " Hostname   : $(hostname)"
log "===== "

log "\n=== 1. SYSTEM IDENTITY ==="
hostnamectl | tee -a "$REPORT"

log "\n=== 2. CURRENT USER & PRIVILEGES ==="
log "User       : $(whoami)"
log "ID info    : $(id)"

log "\n=== 3. SUDO PERMISSIONS - CURRENT USER ==="
sudo -l 2>&1 | tee -a "$REPORT"

log "\n=== 4. LOCAL USER ACCOUNTS (UID >= 1000) ==="
awk -F: '$3 >= 1000 {
    printf "%-15s UID=%-6s HOME=%-20s SHELL=%s\n",
        $1,$3,$6,$7
}' /etc/passwd | tee -a "$REPORT"

log "\n=== 5. SUDO GROUP MEMBERS ==="
getent group sudo | tee -a "$REPORT"

log "\n=== 6. SUDO PERMISSIONS - ALL LOCAL USERS ==="
awk -F: '$3 >= 1000 && $3 < 65534 {print $1}' /etc/passwd |
while read user; do
    log "\n--- $user ---"
    sudo -l -U "$user" 2>&1 | tee -a "$REPORT"
done

log "\n=== 7. SSH EFFECTIVE CONFIGURATION ==="
sudo sshd -T |
grep -E '^(permitrootlogin|passwordauthentication|pubkeyauthentication|
kbdinteractiveauthentication|maxauthtries|allowusers|permitemptypasswords|
x11forwarding|port) ' |
tee -a "$REPORT"

log "\n=== 8. SSH SERVICE STATUS ==="
log "Enabled : $(systemctl is-enabled ssh 2>/dev/null || systemctl is-enabled
sshd 2>/dev/null)"
log "Active  : $(systemctl is-active ssh 2>/dev/null || systemctl is-active sshd
2>/dev/null)"

log "\n=== 9. LISTENING PORTS & SERVICES ==="
sudo ss -tlnp | tee -a "$REPORT"

log "\n=== 10. RUNNING SERVICES ==="
systemctl list-units \
    --type=service \
    --state=running \
    --no-pager | tee -a "$REPORT"

log "\n=== 11. FIREWALL STATUS ==="
sudo ufw status verbose 2>&1 | tee -a "$REPORT"

log "\n=== 12. PENDING SECURITY UPDATES ==="
apt list --upgradable 2>/dev/null |

```



```
grep -i security |
tee -a "$REPORT"

log "\n=== 13. FAILED SSH LOGIN ATTEMPTS (last 20) ==="
sudo grep -i 'failed\|invalid' /var/log/auth.log 2>/dev/null |
tail -n 20 |
tee -a "$REPORT"

log "\n=== 14. RECENT SUCCESSFUL LOGINS ==="
last -n 10 | tee -a "$REPORT"

log "\n=== 15. DISK USAGE ==="
df -h | tee -a "$REPORT"

log "\n===== "
log " ASSESSMENT COMPLETE"
log " Report saved to: $REPORT"
log "===== "
```

7.3 Script Tasarım Notları

- Script bilerek read-only tasarlandı.
- Envanter, maruziyet incelemesi ve operasyonel kanıt toplama işlevlerini bir araya getirir.
- `tee -a` kullanıldığı için çıktı hem terminalde görünür hem de rapor dosyasına yazılır.
- Script `sudo` ile çalıştırıldığı için `$HOME` değişkeni `/root` olarak çözüldü ve raporlar root home dizinine yazıldı.

8. Tehdit Modelleme ve Risk Değerlendirmesi

8.1 Varlık Envanteri

Varlık	Açıklama	Güvenlik Önemi
EC2 instance	Bulutta çalışan Ubuntu sunucusu	Yüksek
SSH servisi	Birincil uzaktan yönetim kanalı	Kritik
ubuntu hesabı	Ana yönetici hesabı	Kritik
dev-user hesabı	İkincil yerel kimlik	Orta-Yüksek
UFW kuralları	Host seviyesinde maruziyet kontrolü	Kritik
Nginx servisi	Public HTTP servisi	Orta
SSH yapılandırması	Uzaktan erişim kontrol kümesi	Kritik
Authentication logları	Erişim denemeleri için kanıt kaynağı	Yüksek
Kernel ve paketler	Çekirdek yürütme katmanı	Kritik

8.2 Risk Kaydı

ID	Bulgu	Threat	Control	Validation
R-01	22 portunda public SSH maruziyeti	Brute-force denemeleri, tarama, kimlik bilgisi istismarı	Key-based access'ı korumak, Fail2Ban'i aktif tutmak, password auth'ı kapalı tutmak	Efektif SSH config ve Fail2Ban jail durumunu kontrol etmek
R-02	80 portunda dinleyen Nginx	Gereksiz saldırı yüzeyi, versiyon sızıntısı, gelecekte web tabanlı istismar yolu	Nginx'i durdurmak ve firewall allow kuralını kaldırmak	Servisin inactive olduğunu ve yalnızca 22'nin kaldığını doğrulamak
R-03	X11Forwarding yes	Gereksiz SSH özellik maruziyeti	X11 forwarding'i kapatmak	sshd -T ile tekrar kontrol etmek
R-04	dev-user için aktif parola durumu	Ek bir authentication entry point	Hesabı kilitlemek	Parola durumunun locked olduğunu doğrulamak
R-05	/home/dev-user/.ssh üzerinde zayıf izinler ve yanlış sahiplik	Anahtar kurcalama veya SSH materyaline yetkisiz erişim	chmod 700 uygulamak ve sahipliği düzeltmek	Dizini tekrar listeleyip owner ve mode'u doğrulamak
R-06	EC2 üzerinde çalışan ModemManager	Gereksiz servis footprint'i	Servisi durdurmak ve disable etmek	Servisin active/enabled durumunu kontrol etmek
R-07	Daha sonra görülen kernel ilişkili bekleyen güncellemeler	Patch uygulanmazsa bilinen zafiyetler devam eder	Güncellemeleri uygulamak ve paket durumunu doğrulamak	Post-update paket durumunu ve gerekiyorsa reboot ihtiyacını kontrol etmek

8.3 STRIDE Eşlemesi

STRIDE Kategorisi	Lab İçindeki Örnek	Azaltım
Spoofing	Public SSH üzerinde yetkisiz authentication denemeleri	Key-based authentication, kapalı password auth, Fail2Ban
Tampering	SSH dizininde zayıf sahiplik ve izinler	Kısıtlayıcı mode ve doğru sahiplik
Repudiation		

	Login ve failed-access denemeleri için kanıt ihtiyacı	Authentication log incelemesi ve tekrar üretilebilir baseline raporları
Information Disclosure	Default Nginx servisi ile HTTP varlığının ve versiyon ailesinin açığa çıkması	Servisi kapatmak ve public maruziyeti kaldırmak
Denial of Service	SSH üzerine tekrar eden kötü niyetli bağlantı denemeleri	Fail2Ban ve sınırlı auth denemesi
Elevation of Privilege	Yönetici hesabı istismarı veya güncel olmayan bileşenlerin sömürülmesi	Least privilege incelemesi ve patch management

9. Lab 2 - Hardening Adımları

9.1 X11 Forwarding'i Kapatmak

Önce aktif yapılandırma satırı bulundu:

```
sudo grep -n "X11Forwarding" /etc/ssh/sshd_config
```

Gözlemlenen kanıt:

```
99:X11Forwarding yes
128:# X11Forwarding no
```

Daha sonra aktif satır değiştirildi ve SSH yapılandırması reload öncesi syntax kontrolünden geçirildi:

```
sudo sed -i '99s/yes/no/' /etc/ssh/sshd_config
sudo sshd -t && sudo systemctl reload ssh
sudo sshd -T | grep x11forwarding
```

Doğrulama sonucu:

```
x11forwarding no
```

Bu adım iyi bir güvenli değişiklik yönetimi örneğidir: aktif satırı bulmak, değişikliği yapmak, daemon yapılandırmasını test etmek, güvenli biçimde reload etmek ve efektif sonucu doğrulamak.

9.2 Gereksiz ModemManager Servisini Kapatmak

EC2 instance üzerinde modem yönetimine ihtiyaç olmadığı için servis durdurulup disable edildi:

```
sudo systemctl stop ModemManager && sudo systemctl disable ModemManager
```

Gözlemlenen çıktı:

```
Removed "/etc/systemd/system/dbus-org.freedesktop.ModemManager1.service".
Removed "/etc/systemd/system/multi-user.target.wants/ModemManager.service".
```

Doğrulama için kullanılan komutlar:

```
systemctl is-active ModemManager
systemctl is-enabled ModemManager
```

Gözlemlenen durum içinde şu ifade yer aldı:

```
inactive
```

Böylece host üzerindeki gereksiz servis ayak izi azaltılmış oldu.

9.3 dev-user Hesabını Kilitlemek

Yerel hesap silinmek yerine kilitlendi:

```
sudo usermod -L dev-user  
sudo passwd -S dev-user
```

Doğrulama sonucu:

```
dev-user L 2026-07-30 0 99999 7 -1
```

Durumun P'den L'ye dönmesi, parolanın kilitlendiğini gösterir. Bu yaklaşım, sahiplik ilişkilerini ve izlenebilirliği korurken hesabın parola tabanlı kullanımını engeller.

9.4 .ssh Dizini Sahipliğini ve İzinlerini Düzeltmek

SSH dizini şu komutla incelendi:

```
sudo ls -la /home/dev-user/.ssh/ 2>/dev/null || echo "No .ssh directory found"
```

Gözlemlenen kanıt:

```
total 8  
drwxrwxrwx 2 root      root      4096 Jul 30 12:24 .  
drwxr-x--- 3 dev-user dev-user  4096 Jul 30 12:24 ..
```

Düzeltici komutlar:

```
sudo chmod 700 /home/dev-user/.ssh  
sudo chown dev-user:dev-user /home/dev-user/.ssh
```

Doğrulama:

```
sudo ls -la /home/dev-user/ | grep .ssh
```

Gözlemlenen kanıt:

```
drwx----- 2 dev-user dev-user 4096 Jul 30 12:24 .ssh
```

Böylece hem yetki modeli hem de sahiplik modeli düzeltilmiş oldu.

9.5 Kullanılmayan Nginx Maruziyetini Kaldırmak

Nginx'in aktif olduğu yerel HTTP cevabıyla doğrulanmıştı. Kasıtlı bir web iş yükü bulunmadığı için servis durdurulup disable edildi:

```
sudo systemctl stop nginx && sudo systemctl disable nginx
```

Daha sonra public firewall kuralı kaldırıldı:

```
sudo ufw delete allow 80/tcp
```

Gözlemlenen çıktı:

```
Rule deleted
Rule deleted (v6)
```

Doğrulama için kullanılan komutlar:

```
sudo ufw status numbered
systemctl is-active nginx
```

Gözlemlenen kanıt:

```
Status: active

[ 1] 22/tcp                ALLOW IN    Anywhere
[ 2] 22/tcp (v6)           ALLOW IN    Anywhere (v6)

inactive
```

Böylece host firewall üzerinde kasıtlı olarak erişilebilir bırakılan tek servis SSH oldu.

9.6 Fail2Ban Durumunun Gözden Geçirilmesi

SSH jail durumu şu komutla incelendi:

```
sudo fail2ban-client status sshd
```

Gözlemlenen kanıt:

```
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:    1
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
  |- Currently banned: 0
  |- Total banned:    0
  `-- Banned IP list:
```

Bu çıktı, savunma mekanizmasının aktif olduğunu ve SSH ile ilişkili olaylar gördüğünü, ancak gözlenen zaman aralığında ban eşiğinin aşılmadığını gösterdi.

10. Doğrulama ve Before/After Karşılaştırması

10.1 Baseline Scriptini Yeniden Çalıştırmak

Hardening sonrasında aynı script yeniden çalıştırıldı:

```
sudo ~/ec2_security_baseline_check.sh
```

Bu, metodolojik açıdan kritik bir detaydır. Güvenlik değişiklikleri varsayımla değil, yeniden ölçüm yapılarak doğrulanmalıdır.

10.2 Temel Before/After Değişimleri

Alan	Önce	Sonra
SSH X11 Forwarding	x11forwarding yes	x11forwarding no
HTTP maruziyeti	Nginx 0.0.0.0:80 ve [::]:80 üzerinde expose	80 portu artık UFW üzerinden açık değil ve Nginx inactive
dev-user parola durumu	P (aktif parola)	L (kilitli)
/home/dev-user/.ssh	drwxrwxrwx root root	drwx----- dev-user dev-user
ModemManager	Running / enabled	Stopped / disabled
Firewall rule set	22 ve 80 açık	Yalnızca 22 açık

10.3 Hardening Sonrası Kanıt

Baseline'ın tekrar çalıştırılması, temel hardening adımlarının etkili olduğunu doğruladı.

Firewall aktif kaldı, HTTP allow kuralı kaldırıldı, gereksiz servisler artık aktif değildi ve SSH yapılandırması X11 forwarding değişikliğini yansıtıyordu.

Daha sonraki bir kontrol ayrıca kernel ile ilişkili güncellemelerin yeniden gözden geçirilmesi gerektiğini gösterdi. Bu, vulnerability management tarafında önemli bir gerçeği ortaya koyar: update durumu zamana bağlıdır ve sürekli izlenmelidir.

11. Bulguların Teknik Yorumu

11.1 Nginx Neden Gerçek Bir Güvenlik Bulgusuydu

Bir servis yalnızca varsayılan kurulumun parçası olduğu ya da basit bir sayfa döndürdüğü için güvenli sayılmaz. Eğer public erişilebilir bir servisin işlevsel bir amacı yoksa, o servis

gereksiz saldırı yüzeyidir. Bu örnekte Nginx; HTTP probing, versiyon ailesi fingerprinting'i ve ileride oluşabilecek web tabanlı yanlış yapılandırma riskleri için bir giriş noktasıydı.

11.2 dev-user Hesabı Neden Önemliydi

Sınırlı yetkili veya belgelenmemiş yerel hesaplar bile önemlidir; çünkü kimlik doğrulama yüzeyini genişletirler. Saldırganın fayda elde etmesi için mutlaka tam sudo yetkisine ihtiyacı yoktur. Bu nedenle hesap anlamlı bir kimlik riski olarak ele alındı ve kilitlendi.

11.3 Efektif SSH Yapılandırması Neden Tek Dosya Okumaktan Daha Güvenilirdir

`sshd -T` komutu, SSH servisinin gerçekten uygulayacağı efektif yapılandırmayı gösterir. Bu, yalnızca bir dosyayı okumaktan daha güvenilirdir; çünkü gerçek daemon davranışı include izinleri ya da ek yapılandırma dosyalarından da etkilenebilir.

11.4 Permission Düzeltmeleri Neden Kritikti

`.ssh` dizini, kimlik doğrulama materyali barındırabileceği için güvenlik açısından kritik bir konumdur. World-writable izinler ve yanlış sahiplik; dosya kurcalama, anahtar kötüye kullanımı veya güven modelinin bozulması için uygun koşullar yaratır.

12. Çıkarılan Dersler

- Hardening'den önce envanter çıkarılmalıdır.
- Efektif yapılandırma runtime-aware komutlarla doğrulanmalıdır.
- Kullanılmayan servisler zararsız görünseler bile risktir.
- Kimlik incelemesi, port incelemesi kadar önemlidir.
- İzinler ve sahiplik yalnızca housekeeping değil, erişim kontrolünün bir parçasıdır.
- Güvenlik doğrulaması before/after kanıt izi gerektirir.
- Otomasyon, tekrar eden incelemelerde tutarlılığı ciddi biçimde artırır.

13. Kalan Riskler ve Geliştirme Fırsatları

- 22 portundaki SSH hâlâ public erişime açıktı; ileride source IP kısıtlaması veya AWS Security Group tasarımı ile daraltılabilir.
- `ubuntu` hesabı hâlâ geniş yönetim yetkisine sahipti; operasyonel açıdan kullanışlı olsa da etkisi yüksektir.
- Baseline scripti severity scoring ve daha temiz rapor yolu yönetimi ile geliştirilebilir.
- Merkezi loglama ve SIEM entegrasyonu henüz uygulanmadı.
- File integrity monitoring henüz eklenmedi.
- Host seviyesindeki bu rapor AWS Security Group kurallarını tam olarak kapsamlı biçimde belgelemedi; bulut genelinde ayrı bir review ile tamamlanmalıdır.

14. Önerilen Repository Yapısı

```
project-1-linux-hardening/  
├── README.md  
├── scripts/  
│   └── ec2_security_baseline_check.sh  
├── reports/  
│   ├── baseline-before-hardening.txt  
│   └── baseline-after-hardening.txt  
├── docs/  
│   ├── threat-model.md  
│   ├── hardening-notes.md  
│   └── images/  
│       └── module-03-security-engineering.png
```

15. Sonuç

Lab 1 ve Lab 2 birlikte, Linux güvenlik mühendisliğinin temel ama tam bir iş akışını temsil eder. Sunucu önce güvenli olduğu varsayılan bir sistem gibi değil, anlaşılması gereken bilinmeyen bir sistem gibi ele alındı. Toplanan kanıtlar hem güvenli varsayılanların hem de anlamlı zayıflıkların bir arada bulunduğunu gösterdi: internete açık ve taranan bir SSH servisi, 80 portunda gereksiz yere expose edilmiş bir Nginx servisi, aktif parolaya sahip ek bir yerel kullanıcı, gereksiz bir SSH özelliği ve güvenlik açısından hassas bir dizinde zayıf sahiplik veya izinler.

Bu bulgular hedefli hardening aksiyonlarını yönlendirdi. Sonuçta daha temiz ve savunulabilir bir sunucu duruşu elde edildi: küçülmüş saldırı yüzeyi, daha iyi kimlik kontrolü, daha güvenli SSH davranışı ve daha güçlü host seviyesi hijyen.

Bir sonraki mantıklı faz, bu sertleştirilmiş baseline'ın üzerine loglama, kötüye kullanım tespiti ve olay müdahale simülasyonu inşa etmektir.